Architettura e Configurazione dei Guardiani

Analisi della struttura dati in ROM/RAM, codifica Bit/Flags, tipi di Nemici e gestione del Movimento in Jet Set Willy

In *Jet Set Willy* per ZX Spectrum, la gestione dei nemici (comunemente denominati **Guardiani**) è integrata all'interno della struttura binaria da 286 byte di ogni stanza. Ogni stanza supporta fino a un massimo di **8 guardiani indipendenti** (slot da 5 byte ciascuno in ROM), oltre a due array dedicati alla definizione delle palette colore (4 byte per i nemici 1-4 e 4 byte per i nemici 5-8). Questa guida documenta la codifica binaria, la scelta degli sprite, il comportamento dinamico in RAM ed il flusso di esecuzione delle routine in enemy.asm.

1. Anatomia della Struttura Dati dei Guardiani nella Stanza (Offset Memoria)

All'interno della struttura di 286 byte della stanza (es. BallRoomEastLayout), i guardiani occupano esattamente **48 byte** suddivisi in tre sezioni fisse:

Offset (Byte)	Dimensione	Sezione Dati (Simbolo)	Descrizione e Ruolo nel Gioco
`223 - 226`	4 Byte	**EnemyColors1_4**	Palette colori per i guardiani da Slot 1 a Slot 4 (1 byte di attributo Spectrum per ciascuno).
`240 - 279`	40 Byte	**Enemies 1-8**	8 definizioni di guardiani (5 byte per ciascuno): Flags/Fase, Tipo Sprite, Pos Iniziale, Coordinata Fissa e Limite di Movimento.
`282 - 285`	4 Byte	**EnemyColors5_8**	Palette colori per i guardiani da Slot 5 a Slot 8 (1 byte di attributo Spectrum per ciascuno).

Struttura dei 5 Byte di Configurazione in ROM per Ciascun Guardiano:

Byte ROM	Nome Campo	Formato Bit / Valore	Funzione e Significato Logico
`Byte 0`	**Flags & Fase**	`%A V F1 F0 - - - -`	**Bit 7 (A)**: Active (1=Attivo, 0=Disattivato). **Bit 6 (V)**: Asse (0=Orizzontale, 1=Verticale). **Bit 5-4 (F)**: Fase di spawn iniziale (0..3).
`Byte 1`	**Type (Sprite)**	`%- - T5..T0`	**Bit 5-0**: ID Tipo Sprite Nemico (0..41, es. 0=Lametta, 2=Uccello, 6=Faccia, 13=Canguro, ecc.).
`Byte 2`	**Start Coordinate**	`0 .. 255 pixel`	Coordinata iniziale lungo l'asse dinamico (**StartX** se Orizzontale, **StartY** se Verticale).
`Byte 3`	**Fixed Coordinate**	`0 .. 255 pixel`	Coordinata fissa lungo l'asse ortogonale (**StartY** se Orizzontale, **StartX** se Verticale).
`Byte 4`	**Limit Coordinate**	`0 .. 255 pixel`	Coordinata di fine corsa lungo l'asse dinamico. Insieme a Start forma l'intervallo [MinLimit, MaxLimit].

2. Codifica del Byte 0 (Flags, Asse e Fase Iniziale)

Il Byte 0 controlla l'attivazione del nemico, l'asse di movimento e la sua posizione/direzione iniziale di spawn:

- **Bit 7 — Stato Attivazione (Active Flag): 1** (≥ $80 / 128) = Attivo (aggiornato e renderizzato); **0** (< $80) = Inattivo (slot disattivato).
- **Bit 6 — Asse di Movimento (Movement Axis): 0** ($00) = Movimento **Orizzontale** (Y fissa); **1** ($40 / +64) = Movimento **Verticale** (X fissa).
- **Bit 5-4 — Fase Iniziale di Spawn e Direzione (Phase Bits):** permettono di sfasare più nemici sulla stessa traiettoria:

Bit 5-4	Valore Hex Flag	Fase	Posizione Iniziale di Calcolo in RAM	Direzione Iniziale
`%00`	`$80 / $C0`	**Fase 0**	Parte dalla coordinata iniziale impostata nel Byte 2 (Start).	Destra / Basso (Dir = 2)
`%01`	`$90 / $D0`	**Fase 1**	Parte a 1/4 del percorso: MinLimit + (Delta / 4).	Destra / Basso (Dir = 2)
`%10`	`$A0 / $E0`	**Fase 2**	Parte a metà percorso: MinLimit + (Delta / 2).	Sinistra / Alto (Dir = 1)
`%11`	`$B0 / $F0`	**Fase 3**	Parte dal limite massimo: MaxLimit.	Sinistra / Alto (Dir = 1)

3. Tabella dei Tipi di Guardiani (Byte 1 — Sprite ID 0..41)

Il motore di gioco include **42 tipi di sprite differenti** definiti in constants.asm e renderizzati da Enemy_DrawRoom:

ID	Costante ASM	Nome / Descrizione	Animazione & Note Direzionali
`0`	`LAMETTA`	Lametta / Razor	Sprite direzionale (Left / Right), 4 frame.

1	SEGA	Sega da falegname	Sprite direzionale (Left / Right), 4 frame.
2	UCCELLO	Uccello volante	Sprite direzionale (Left / Right), 4 frame.
3	RADAR	Radar rotante	Omnidirezionale, 4 frame animati.
4	UOVO	Uovo camminante	Sprite direzionale (Left / Right), 4 frame.
5	UFO	Disco volante UFO	Omnidirezionale, 4 frame animati.
6	FACCIA	Faccia sorridente	Omnidirezionale, 4 frame animati.
7	SCIMMIA	Scimmia	Omnidirezionale, 4 frame animati.
8	ROBOT	Robot da cucina	Omnidirezionale, 4 frame animati.
9	PALLA	Palla che rimbalza	Omnidirezionale, 4 frame animati.
10	DOUBLE_BALL	Doppia palla	Sprite direzionale (Left / Right), 4 frame.
11	SOLDATO	Guardia / Soldato	Animazione ridotta a 2 frame (0..1).
12	MAIALE	Maiale volante	Sprite direzionale (Left / Right), 4 frame.
13	CANGURO	Canguro saltellante	Sprite direzionale, animazione a 8 frame (0..7).
14	FIORE	Fiore / Pianta	Omnidirezionale, 4 frame animati.
15	FACCIA_PALLA	Faccia Palla	Omnidirezionale, 4 frame animati.
16	CANDELABRO	Candelabro acceso	Animazione ridotta a 2 frame (0..1).
17	FRECCIA	Freccia indicatrice	Sprite direzionale (Left / Right).
18..22	TRONCHESE..FRATE	Utensili / Nonnino / Frate	Varianti animate (es. Frate con 2 direzioni).
23..26	FACE_HL..FACE_LR	Faccia Gigante 32x32	Sprite statico in 4 quadranti 16x16 (HL, HR, LL, LR).
27..31	PINZA..PINGUINO	Gelato / Pinguino / Ruota	Ostacoli e guardiani tematici.
32..41	MURO..PALAZZO	Piede / Diavolo / Botte ecc.	Nemici speciali e boss ambientali.

4. Struttura Dati Dinamica in RAM (ActiveEnemies)

All'ingresso in una stanza, Enemy_LoadRoom alloca l'array ActiveEnemies in RAM (8 slot × 11 byte = **88 byte totali**):

Offset RAM	Campo	Tipo Dati	Descrizione Logica e Utilizzo nel Frame Loop
Offset 0	**Flags**	Byte	Copia del Byte 0 della ROM (Active, Vertical, Phase).
Offset 1	**Sprite Type**	Byte	Copia del Byte 1 della ROM (Tipo nemico 0..41).
Offset 2	**Current X**	Byte (Pixel)	Coordinata X corrente a schermo (aggiornata di ±2px per frame se orizzontale).
Offset 3	**Current Y**	Byte (Pixel)	Coordinata Y corrente a schermo (aggiornata di ±2px per frame se verticale).
Offset 4	**Min Limit**	Byte (Pixel)	Calcolato come min(Start, Limit). Limite inferiore/sinistro del percorso.
Offset 5	**Max Limit**	Byte (Pixel)	Calcolato come max(Start, Limit). Limite superiore/destro del percorso.
Offset 6	**Direction**	Byte (1 o 2)	**1** = Sinistra/Alto (decrementa coord), **2** = Destra/Basso (incrementa coord).
Offset 7	**Anim State**	Byte (0..3/7)	Contatore del frame d'animazione corrente, incrementato ad ogni movimento.
Offset 8	**Old X**	Byte (Pixel)	Coordinata X del frame precedente, usata da Enemy_EraseRoom per pulire lo schermo.
Offset 9	**Old Y**	Byte (Pixel)	Coordinata Y del frame precedente, usata da Enemy_EraseRoom per pulire lo schermo.
Offset 10	**Color**	Byte (Attr)	Byte attributo colore ZX Spectrum recuperato da EnemyColors1_4 o EnemyColors5_8.

5. Gestione dei Colori ed Attributi Spectrum

I colori dei guardiani sono definiti in due array separati da 4 byte nella stanza:

• **Offset 223 - 226 (EnemyColors1_4):** Colori per Slot 1..4. • **Offset 282 - 285 (EnemyColors5_8):** Colori per Slot 5..8.

Formato attributo Spectrum: FLASH(1b) | BRIGHT(1b) | PAPER(3b) | INK(3b). Costanti da constants.asm:

Costante Standard	Hex	Costante FLASH	Hex	Colore Risultante a Schermo
NONE	$00	—	—	Nero / Trasparente (PAPER 0, INK 0)
BLUE	$01	BLUE_FLASH	$C1	Blu (Standard / Lampeggiante)
RED	$02	RED_FLASH	$C2	Rosso (Standard / Lampeggiante)
MAGENTA	$03	MAGENTA_FLASH	$C3	Magenta / Viola (Standard / Flash)

GREEN	$04	GREEN_FLASH	$C4	Verde (Standard / Flash)
CYAN	$05	CYAN_FLASH	$C5	Ciano / Azzurro (Standard / Flash)
YELLOW	$06	YELLOW_FLASH	$C6	Giallo (Standard / Flash)
WHITE	$07	WHITE_FLASH	$C7	Bianco (Standard / Flash)

6. Esempio Pratico di Configurazione Assembly (BallRoomEast.asm)

Estratto dal file rooms/BallRoomEast.asm che illustra la dichiarazione di colori e guardiani:

```
; --- Palette Colori Nemici 1-4 (Offset 223) ---
BallRoomEastReserved:
    DEFB NONE, MAGENTA, CYAN, YELLOW     ; Nemico 1 (Nero), Nemico 2 (Magenta), Nemico 3 (Ciano), Nemico 4 (Giall
o)

; --- Definizioni dei 8 Nemici (Offset 240, 5 byte ciascuno) ---
BallRoomEastEnemy1: DEFB 0, 0, 0, 0, 0                   ; Slot 1: Disattivato (Byte 0 = 0)
BallRoomEastEnemy2: DEFB $C0, FACCIA, 40, 48, 87         ; Slot 2: Vert, Phase 0, Faccia, Y=40, X=48, LimitY=87
BallRoomEastEnemy3: DEFB $E0, FACCIA, 56, 48, 96         ; Slot 3: Vert, Phase 2, Faccia, Y=56, X=48, LimitY=96
BallRoomEastEnemy4: DEFB $C0, FACCIA, 88, 72, 96         ; Slot 4: Vert, Phase 0, Faccia, Y=88, X=72, LimitY=96
BallRoomEastEnemy5: DEFB $80, UCCELLO, 112, 80, 232       ; Slot 5: Horiz, Phase 0, Uccello, X=112, Y=80, LimitX=
232
BallRoomEastEnemy6: DEFB 0, 0, 0, 0, 0                   ; Slot 6: Disattivato
BallRoomEastEnemy7: DEFB 0, 0, 0, 0, 0                   ; Slot 7: Disattivato
BallRoomEastEnemy8: DEFB 0, 0, 0, 0, 0                   ; Slot 8: Disattivato

; --- Palette Colori Nemici 5-8 (Offset 282) ---
BallRoomEastEnemyColors5_8:
    DEFB MAGENTA, NONE, NONE, NONE     ; Nemico 5 (Magenta), Nemico 6-8 (Nessuno)
```

- **Guardiano 2 (Faccia Vert):** Flag $C0 (%11000000 -> Active=1, Vert=1, Phase=0). Parte da Y=40 con X=48 e oscilla in verticale fino a Y=87.
- **Guardiano 3 (Faccia Vert Sfasata):** Flag $E0 (%11100000 -> Active=1, Vert=1, Phase=2). Parte a metà percorso (Y=56) verso l'alto (Dir=1), sfasando perfettamente il movimento dal Nemico 2.
- **Guardiano 5 (Uccello Horiz):** Flag $80 (%10000000 -> Active=1, Horiz=0, Phase=0). Parte da X=112 a Y=80 e vola orizzontalmente fino a X=232.

7. Ciclo di Vita del Motore di Gioco (enemy.asm)

Il rendering ed aggiornamento dei guardiani avviene in quattro fasi ad ogni frame di gioco:

Passo	Routine / Funzione	Operazione Eseguita nel Motore di Gioco
1	**Enemy_LoadRoom**	Inizializzazione all'ingresso della stanza. Legge i 5 byte ROM per ciascuno dei 8 nemici, imposta ActiveEnemies in RAM, calcola i limiti MinLimit/MaxLimit, imposta la fase/direzione iniziale e copia il colore corrispondente.
2	**Enemy_EraseRoom**	Prima di spostare i guardiani, cancella la grafica dallo schermo usando le coordinate memorizzate in Old X / Old Y, ripristinando le tile dello sfondo ed i relativi attributi di colore.
3	**Enemy_Update**	Verifica il divisore di velocità (enemy_update_rate). Salva Current X/Y in Old X/Y, avanza di 2 pixel lungo l'asse attivo, inverte la direzione ai limiti MinLimit/MaxLimit ed incrementa l'animazione (Anim State).
4	**Enemy_DrawRoom**	Scorre l'array ActiveEnemies in RAM. Recupera l'indirizzo della bitmap dello sprite 16×16 in base a Tipo, Direzione (Left/Right) ed Anim State. Disegna i pixel a schermo e applica l'attributo colore ZX Spectrum nella RAM attributi ($5800).